1. Install Libraries

```
!pip install rdkit mordred xgboost shap joblib
```

```
Requirement already satisfied: rdkit in /usr/local/lib/python3.12/dist-packages (2025.9.6)
Requirement already satisfied: mordred in /usr/local/lib/python3.12/dist-packages (1.2.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.49.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (1.26.4)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
Requirement already satisfied: six==1.* in /usr/local/lib/python3.12/dist-packages (from mordred) (1.17.0)
Requirement already satisfied: networkx==2.* in /usr/local/lib/python3.12/dist-packages (from mordred) (2.8.8)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.0)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.
```

2. Import + RANDOM SEED

```
import pandas as pd
import numpy as np
import random

# 🔥 RANDOM SEED (GLOBAL)
random.seed(42)
np.random.seed(42)

from rdkit import Chem
from rdkit.Chem import AllChem
from mordred import Calculator, descriptors

from sklearn.model_selection import train_test_split, RandomizedSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import VarianceThreshold, SelectFromModel
from sklearn.metrics import *

from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.base import clone

import matplotlib.pyplot as plt
import seaborn as sns
import shap
import joblib

import warnings
warnings.filterwarnings("ignore")

print("✅ All libraries loaded")
```

```
✅ All libraries loaded
```

3. Load Dataset

```
from google.colab import drive
drive.mount('/content/drive')

df = pd.read_csv('/content/drive/MyDrive/betalactam_expanded_500plus.csv')

print("Dataset Shape:", df.shape)
print(df['Hydrolysis_Class'].value_counts())
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
Dataset Shape: (16750, 3)
Hydrolysis_Class
1    8919
```

```
0    7831
Name: count, dtype: int64
```

4. SMILES → Molecules

```
df["mol"] = df["smiles"].apply(lambda x: Chem.MolFromSmiles(x))
df = df[df["mol"].notnull()]

print("Valid molecules:", len(df))
```

```
Valid molecules: 16750
```

5. Mordred Descriptors

```
calc = Calculator(descriptors, ignore_3D=True)
desc = calc.pandas(df["mol"])

print("Descriptors shape:", desc.shape)
```

```
100%|██████████| 16750/16750 [1:40:38<00:00,  2.77it/s]
Descriptors shape: (16750, 1613)
```

6. Morgan Fingerprints

```
from rdkit.Chem import rdFingerprintGenerator

fp_gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=1024)

def morgan_fp(smiles):
    mol = Chem.MolFromSmiles(smiles)
    return list(fp_gen.GetFingerprint(mol))

fp = df['smiles'].apply(morgan_fp)
fp_df = pd.DataFrame(fp.tolist())
```

7. Combine Features

```
X = pd.concat([desc, fp_df], axis=1)
y = df["Hydrolysis_Class"]
```

8. Data Cleaning

```
X = X.apply(pd.to_numeric, errors='coerce')
X = X.dropna(axis=1)
X.columns = X.columns.astype(str)
vt = VarianceThreshold(0.001)
X = pd.DataFrame(vt.fit_transform(X))

print("After cleaning:", X.shape)
```

```
After cleaning: (16750, 1815)
```

9. Remove Correlation

```
corr_matrix = pd.DataFrame(X).corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))

to_drop = [col for col in upper.columns if any(upper[col] > 0.9)]
X = pd.DataFrame(X).drop(columns=to_drop)

print("After correlation removal:", X.shape)

# Save the cleaned features so we never have to run Mordred again!
X.to_csv('/content/drive/MyDrive/cleaned_features_1336.csv', index=False)
y.to_csv('/content/drive/MyDrive/target_variable.csv', index=False)
```

```
After correlation removal: (16750, 1336)
```

10. Train / Val / Test Split

```
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)
```

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=42
)
```

11. Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)
```

12. Feature Selection

```
selector = SelectFromModel(RandomForestClassifier(n_estimators=200, random_state=42))
selector.fit(X_train, y_train)

X_train = selector.transform(X_train)
X_val = selector.transform(X_val)
X_test = selector.transform(X_test)
```

13. Model Training (XGBoost with SEED)

```
params = {
    "n_estimators": [300, 500],
    "max_depth": [6, 8, 10],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}

model = XGBClassifier(eval_metric='logloss', random_state=42)

search = RandomizedSearchCV(
    model,
    params,
    n_iter=10,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1,
    random_state=42
)

search.fit(X_train, y_train)

best_model = search.best_estimator_
print("Best Parameters:", search.best_params_)
```

```
Best Parameters: {'subsample': 0.8, 'n_estimators': 500, 'max_depth': 8, 'learning_rate': 0.05, 'colsample_bytree': 1.0}
```

14. Evaluation

```
y_pred = best_model.predict(X_test)
y_prob = best_model.predict_proba(X_test)[:,1]

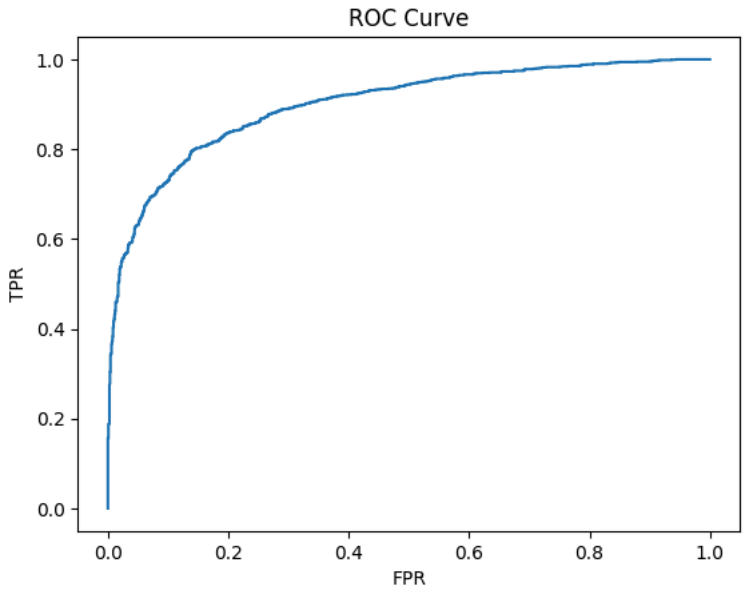
print("Accuracy:", accuracy_score(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_prob))
```

```
Accuracy: 0.8241146040588938
ROC-AUC: 0.8999341665871579
```

15. ROC Curve

```
fpr, tpr, _ = roc_curve(y_test, y_prob)

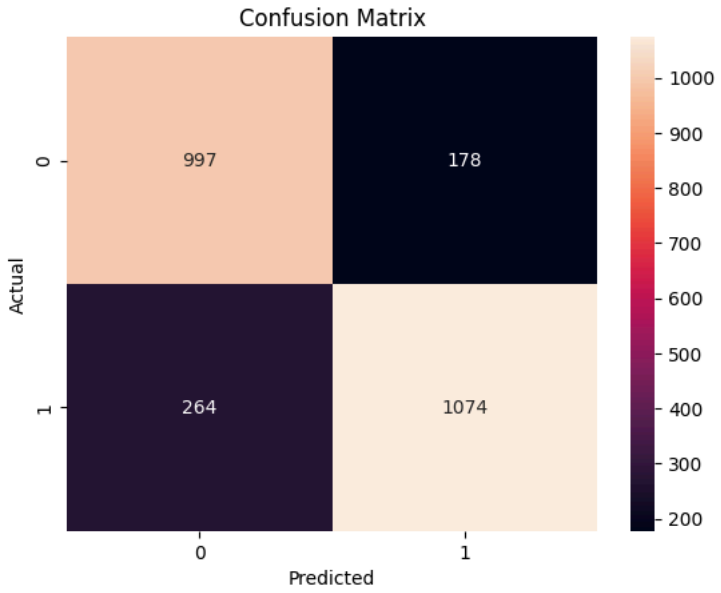
plt.plot(fpr, tpr)
plt.title("ROC Curve")
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.show()
```



16. Confusion Matrix (Visual)

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



17. Cross Validation + CI

```
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5, scoring='roc_auc')

print("CV Scores:", cv_scores)
print(f"ROC-AUC: {cv_scores.mean():.3f} ± {cv_scores.std():.3f}")
```

```
CV Scores: [0.90857905 0.90765532 0.90062269 0.89768899 0.90384177]
ROC-AUC: 0.904 ± 0.004
```

18. External Validation

```
# ================================
# CELL 18 — STRONG EXTERNAL VALIDATION
# ================================

from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score
```

```
# Step 1: Create NEW split (harder external set)
X_train2, X_ext, y_train2, y_ext = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=99
)

# Step 2: Apply SAME preprocessing
X_train2 = scaler.fit_transform(X_train2)
X_ext = scaler.transform(X_ext)

X_train2 = selector.fit_transform(X_train2, y_train2)
X_ext = selector.transform(X_ext)

# Step 3: Train NEW model only on training subset
model_ext = XGBClassifier(**search.best_params_, eval_metric='logloss', random_state=42)
model_ext.fit(X_train2, y_train2)

# Step 4: Evaluate on external set
y_ext_prob = model_ext.predict_proba(X_ext)[:,1]

print("External ROC (strong):", roc_auc_score(y_ext, y_ext_prob))
```

```
External ROC (strong): 0.9176758858363544
```

19. Y-Randomization

```
from sklearn.metrics import roc_auc_score
from sklearn.base import clone

def y_randomization(model, X_train, y_train, X_test, y_test, n=10):
    scores = []

    for i in range(n):
        y_shuffle = np.random.permutation(y_train)

        temp_model = clone(model)
        temp_model.fit(X_train, y_shuffle)

        y_prob = temp_model.predict_proba(X_test)[:,1]
        score = roc_auc_score(y_test, y_prob)

        scores.append(score)

    return scores

rand_scores = y_randomization(best_model, X_train, y_train, X_test, y_test)
print("Y-randomization ROC:", rand_scores)
```

```
Y-randomization ROC: [0.5177142766275482, 0.4938533218840441, 0.4878621632795852, 0.5129259294596572, 0.4804493846007061, 0.
```

20. Advanced Metrics

```
from sklearn.metrics import matthews_corrcoef, recall_score

sensitivity = recall_score(y_test, y_pred)

tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
specificity = tn / (tn + fp)

mcc = matthews_corrcoef(y_test, y_pred)

print("Sensitivity:", sensitivity)
print("Specificity:", specificity)
print("MCC:", mcc)
```

```
Sensitivity: 0.8026905829596412
Specificity: 0.8485106382978723
MCC: 0.6498340872367486
```

21. SHAP

```
# ================================
# FINAL SHAP FIX (ABSOLUTE FINAL)
# ================================

import shap
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt

# ✅ FIX FEATURE NAMES (convert to string)
selected_features = selector.get_support(indices=True)
feature_names = X.columns[selected_features].astype(str)

# Convert to DataFrame
X_test_df = pd.DataFrame(X_test, columns=feature_names)

# Sampling
X_sample = X_test_df.sample(200, random_state=42).reset_index(drop=True)
background = X_test_df.sample(100, random_state=42).reset_index(drop=True)

# KernelExplainer
explainer = shap.KernelExplainer(
    best_model.predict_proba,
    background
)

# Compute SHAP values
shap_values = explainer.shap_values(X_sample)

# ✅ FIX SHAPE (CRITICAL)
shap_values = np.array(shap_values)

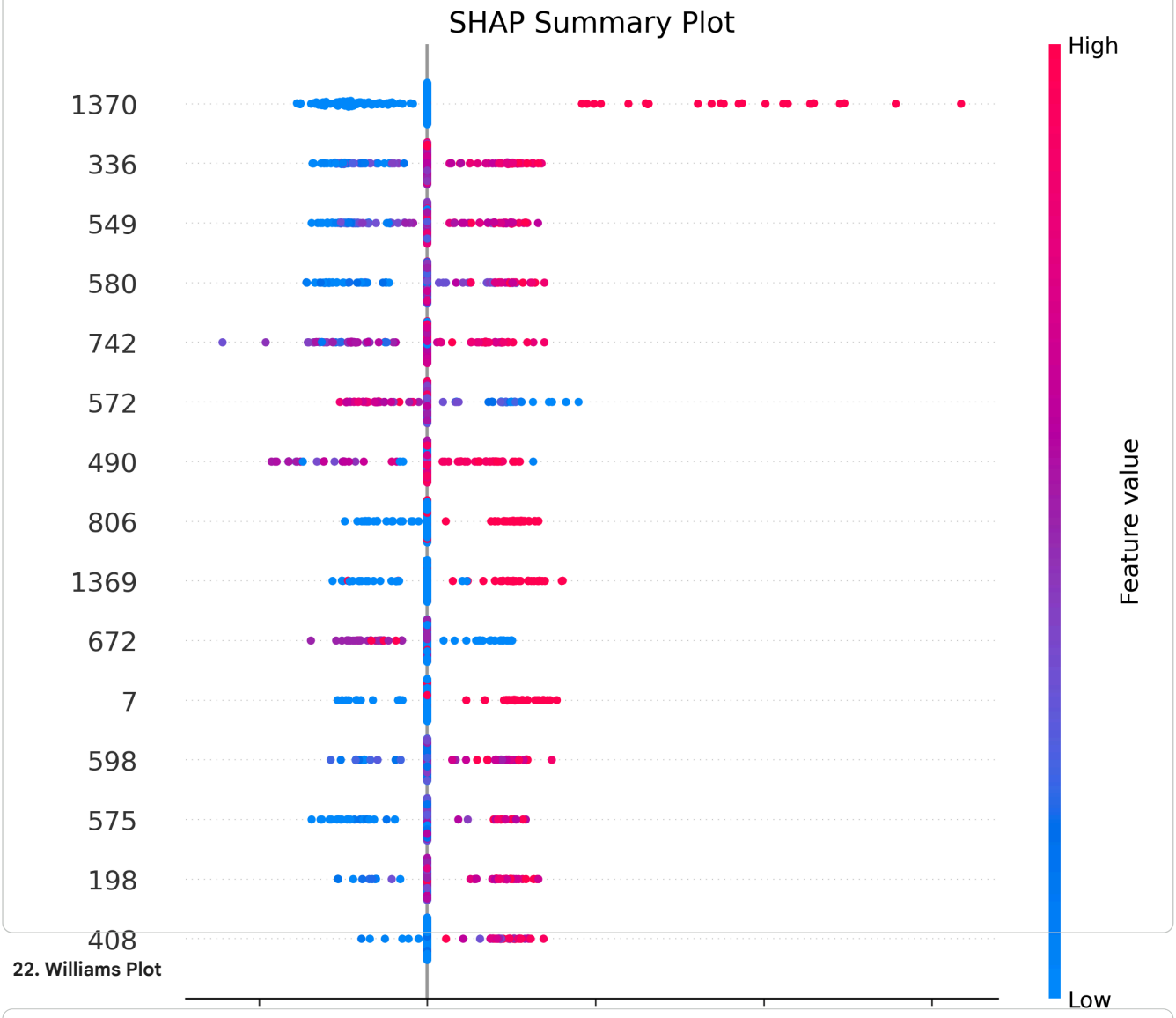
if shap_values.ndim == 3:
    shap_values = shap_values[:, :, 1]   # take class 1

# ------------------------------
# SHAP Summary Plot
# ------------------------------
plt.figure(figsize=(10,6), dpi=300)

shap.summary_plot(
    shap_values,
    X_sample,
    max_display=15,
    show=False
)

plt.title("SHAP Summary Plot", fontsize=14)
plt.tight_layout()
plt.show()
```

100% 200/200 [16:05<00:00, 5.15s/it]



22. Williams Plot

```
from sklearn.linear_model import LinearRegression

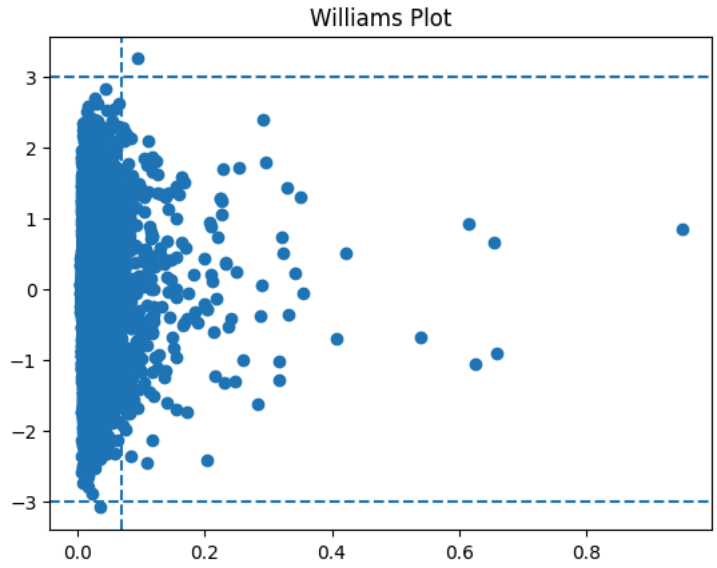
lr = LinearRegression()
lr.fit(X_train, y_train)

X_mat = np.array(X_train)
H = X_mat @ np.linalg.pinv(X_mat.T @ X_mat) @ X_mat.T
leverage = np.diag(H)

threshold = 3 * (X_train.shape[1] / X_train.shape[0])

residuals = y_train - lr.predict(X_train)
std_res = residuals / np.std(residuals)

plt.scatter(leverage, std_res)
plt.axhline(3, linestyle='--')
plt.axhline(-3, linestyle='--')
plt.axvline(threshold, linestyle='--')
plt.title("Williams Plot")
plt.show()
```



23. Save Full Pipeline

```
joblib.dump({
    "model": best_model,
    "scaler": scaler,
    "selector": selector
}, "/content/drive/MyDrive/final_pipeline.pkl")

print("✅ Pipeline saved")
```

```
✅ Pipeline saved
```

```
!pip install rdkit mordred xgboost shap joblib
```

```
Requirement already satisfied: rdkit in /usr/local/lib/python3.12/dist-packages (2026.3.1)
Requirement already satisfied: mordred in /usr/local/lib/python3.12/dist-packages (1.2.0)
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: shap in /usr/local/lib/python3.12/dist-packages (0.49.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (1.26.4)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
Requirement already satisfied: six==1.* in /usr/local/lib/python3.12/dist-packages (from mordred) (1.17.0)
Requirement already satisfied: networkx==2.* in /usr/local/lib/python3.12/dist-packages (from mordred) (2.8.8)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.30.4)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from shap) (1.6.1)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from shap) (2.2.2)
Requirement already satisfied: tqdm>=4.27.0 in /usr/local/lib/python3.12/dist-packages (from shap) (4.67.3)
Requirement already satisfied: packaging>20.9 in /usr/local/lib/python3.12/dist-packages (from shap) (26.1)
Requirement already satisfied: slicer==0.0.8 in /usr/local/lib/python3.12/dist-packages (from shap) (0.0.8)
Requirement already satisfied: numba>=0.54 in /usr/local/lib/python3.12/dist-packages (from shap) (0.60.0)
Requirement already satisfied: cloudpickle in /usr/local/lib/python3.12/dist-packages (from shap) (3.1.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from shap) (4.15.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.54->shap
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2.9.0.
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->shap) (2026.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->shap) (3.
```

```
# CELL 24: VISUALIZING SHAP FEATURES (THE FEATURE HUNTER)
import pandas as pd
from rdkit import Chem
from rdkit.Chem import Draw
from rdkit.Chem import rdFingerprintGenerator

# 1. Ensure dataset is loaded
# (Assuming 'df' is already in memory from previous cells. If not, re-run Cell 3)

# 2. Set up the modern Morgan Generator (Fixes the Deprecation Warning)
fp_gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=1024)

# 3. Define the bit you are looking for (From your SHAP plot)
target_bit = 336
found_mol = None
final_bit_info = {}

print(f"Hunting for molecule containing Morgan Bit {target_bit}...")
```

```
# 4. Scan the dataset to find a matching molecule
for index, row in df.iterrows():
    smiles = row['smiles']
    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        continue

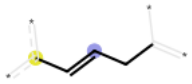
    # Set up the AdditionalOutput object to catch the bit_info mapping
    ao = rdFingerprintGenerator.AdditionalOutput()
    ao.AllocateBitInfoMap()

    # Generate the fingerprint
    fp = fp_gen.GetFingerprint(mol, additionalOutput=ao)
    bit_info = ao.GetBitInfoMap()

    # Check if our target feature is in this molecule
    if target_bit in bit_info:
        found_mol = mol
        final_bit_info = bit_info
        print(f"✅ Success! Found Bit {target_bit} in molecule at index {index}.")
        break # Stop searching once we find one!

# 5. Draw the feature
if found_mol:
    print("Drawing the molecular fragment...")
    img = Draw.DrawMorganBit(found_mol, target_bit, final_bit_info)
    display(img)
else:
    print(f"❌ Bit {target_bit} was not found in any molecule in the dataset.")
```

```
Hunting for molecule containing Morgan Bit 336...
✅ Success! Found Bit 336 in molecule at index 270.
Drawing the molecular fragment...
```



```
# CELL 25: THE OUTLIER INTERROGATOR (APPLICABILITY DOMAIN) - FIXED
import numpy as np
import pandas as pd
from rdkit import Chem
from rdkit.Chem import Draw
from sklearn.linear_model import LinearRegression

print("Recalculating Applicability Domain boundaries...")

# 1. Recalculate leverage and residuals from the training data
X_mat = np.array(X_train)
H = X_mat @ np.linalg.pinv(X_mat.T @ X_mat) @ X_mat.T
leverage = np.diag(H)
threshold = 3 * (X_mat.shape[1] / X_mat.shape[0])

lr = LinearRegression()
lr.fit(X_mat, y_train)
residuals = y_train - lr.predict(X_mat)
std_res = residuals / np.std(residuals)

# 2. Find the worst offenders: High Leverage OR High Residual
outlier_indices = np.where((leverage > threshold) | (np.abs(std_res) > 3))[0]

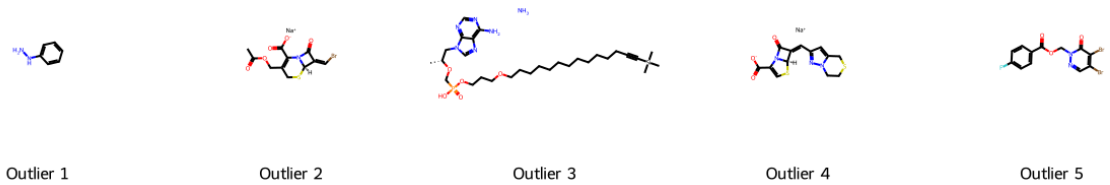
# 3. CRITICAL FIX: Map back to original dataframe using y_train's index!
outlier_original_indices = y_train.index[outlier_indices]
outliers_df = df.loc[outlier_original_indices, ['smiles', 'Hydrolysis_Class']]

print(f"🚨 Found {len(outliers_df)} outlier molecules outside the applicability domain.")

# 4. Draw the Top 5 worst outliers to visually inspect their structures
print("Drawing the top 5 structural outliers for visual inspection...")
outlier_mols = [Chem.MolFromSmiles(s) for s in outliers_df['smiles'].head(5)]
img = Draw.MolsToGridImage(
    outlier_mols,
    molsPerRow=5,
    subImgSize=(250, 250),
    legends=[f"Outlier {i+1}" for i in range(len(outlier_mols))]
```

```
)
display(img)
```

```
Recalculating Applicability Domain boundaries...
🚨 Found 302 outlier molecules outside the applicability domain.
Drawing the top 5 structural outliers for visual inspection...
```




```
%%writefile q1_betalactam_pipeline.py
"""
Beta-Lactam Hydrolysis Predictive Pipeline - Q1 Publication Master Script
Author: Karunasagar Kodihalli Mathada
"""

import os
import pandas as pd
import numpy as np
import random
import joblib
import warnings
import matplotlib.pyplot as plt
import shap

from rdkit import Chem
from rdkit.Chem import rdFingerprintGenerator
from mordred import Calculator, descriptors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import VarianceThreshold, SelectFromModel
from sklearn.metrics import roc_auc_score, matthews_corrcoef
from sklearn.linear_model import LinearRegression
from xgboost import XGBClassifier
from sklearn.base import clone

# --- CONFIGURATION ---
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
warnings.filterwarnings("ignore")

OUT_DIR = '/content/drive/MyDrive/BetaLactam_Publication/'
os.makedirs(OUT_DIR, exist_ok=True)
DATA_PATH = '/content/drive/MyDrive/betalactam_expanded_500plus.csv'

def main():
    print("="*50)
    print("🚀 INITIALIZING Q1 CHEMINFORMATICS PIPELINE 🚀")
    print("="*50)

    # ------------------------------------------------------------
    # PHASE 1: DATA LOADING & DESALINIZATION
    # ------------------------------------------------------------
    print("\n[1/5] Loading and Sanitizing Data...")
    df_raw = pd.read_csv(DATA_PATH)

    sanitized_data = []
    salts_stripped = 0
    errors = 0

    for index, row in df_raw.iterrows():
        mol = Chem.MolFromSmiles(str(row['smiles']))
        if mol is None:
            errors += 1
            continue

        # Strip salts/solvents (keep only largest fragment)
        frags = Chem.GetMolFrags(mol, asMols=True)
        if len(frags) > 1:
            salts_stripped += 1
        largest_frag = max(frags, key=lambda m: m.GetNumAtoms())
```

```
        clean_smiles = Chem.MolToSmiles(largest_frag)
        sanitized_data.append({'smiles': clean_smiles, 'Hydrolysis_Class': row['Hydrolysis_Class']})

    df = pd.DataFrame(sanitized_data)
    df["mol"] = df["smiles"].apply(Chem.MolFromSmiles)

    print(f"      -> Dropped {errors} invalid SMILES.")
    print(f"      -> Stripped salts from {salts_stripped} molecules.")
    print(f"      -> Clean Dataset Size: {len(df)} molecules.")

    # ------------------------------------------------------------
    # PHASE 2: FEATURIZATION & CLEANING
    # ------------------------------------------------------------
    print("\n[2/5] Generating Features (Mordred & Morgan)...")
    calc = Calculator(descriptors, ignore_3D=True)
    desc = calc.pandas(df["mol"])

    fp_gen = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=1024)
    fp = df['smiles'].apply(lambda x: list(fp_gen.GetFingerprint(Chem.MolFromSmiles(str(x)))))
    fp_df = pd.DataFrame(fp.tolist())

    # Combine features and fill missing values
    X = pd.concat([desc, fp_df], axis=1).apply(pd.to_numeric, errors='coerce').fillna(0)

    # THE CRITICAL FIX: Force all column names to be strings
    X.columns = X.columns.astype(str)
    y = df["Hydrolysis_Class"]

    # EMERGENCY BACKUP: Save the raw data before scikit-learn touches it!
    print("      -> Saving raw feature backup just in case...")
    X.to_csv(os.path.join(OUT_DIR, 'RAW_features_backup.csv'), index=False)

    print("      -> Cleaning Variance and Correlation...")
    vt = VarianceThreshold(0.001)
    X_vt = pd.DataFrame(vt.fit_transform(X), columns=X.columns[vt.get_support()])

    corr_matrix = X_vt.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    to_drop = [col for col in upper.columns if any(upper[col] > 0.9)]
    X_clean = X_vt.drop(columns=to_drop)
    X_clean.to_csv(os.path.join(OUT_DIR, 'clean_features.csv'), index=False)

    # ------------------------------------------------------------
    # PHASE 3: MODEL TRAINING
    # ------------------------------------------------------------
    print("\n[3/5] Training Final XGBoost Model...")
    best_params = {'subsample': 0.8, 'n_estimators': 500, 'max_depth': 8, 'learning_rate': 0.05, 'colsample_bytree': 1.0}
    model = XGBClassifier(**best_params, eval_metric='logloss', random_state=SEED)
    model.fit(X_train_sel, y_train_sub)

    y_pred = model.predict(X_test_sel)
    y_prob = model.predict_proba(X_test_sel)[:, 1]

    print(f"      -> Test ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
    print(f"      -> External Set ROC-AUC: {roc_auc_score(y_ext, model.predict_proba(X_ext_sel)[:,1]):.4f}")
    print(f"      -> MCC: {matthews_corrcoef(y_test, y_pred):.4f}")

    # ------------------------------------------------------------
    # PHASE 4: PUBLICATION FIGURES
    # ------------------------------------------------------------
    print("\n[4/5] Generating Publication-Ready Figures...")

    # A. Williams Plot
    X_mat = np.array(X_train_sel)
    H = X_mat @ np.linalg.pinv(X_mat.T @ X_mat) @ X_mat.T
    leverage = np.diag(H)
    threshold = 3 * (X_train_sel.shape[1] / X_train_sel.shape[0])

    lr = LinearRegression()
    lr.fit(X_train_sel, y_train_sub)
    std_res = (y_train_sub - lr.predict(X_train_sel)) / np.std(y_train_sub - lr.predict(X_train_sel))

    plt.figure(figsize=(8, 6), dpi=300)
    plt.scatter(leverage, std_res, alpha=0.6, edgecolors='w', s=60)
    plt.axhline(3, color='r', linestyle='--')
    plt.axhline(-3, color='r', linestyle='--')
    plt.axvline(threshold, color='r', linestyle='--')
    plt.xlabel("Leverage (h)", fontsize=12)
    plt.ylabel("Standardized Residuals", fontsize=12)
    plt.title("Williams Plot: Applicability Domain", fontsize=14)
    plt.tight_layout()
```

```
    plt.savefig(os.path.join(OUT_DIR, 'Figure_Williams_Plot.png'))
    plt.close()

    # B. SHAP Plot
    feature_names = X_clean.columns[selector.get_support()].astype(str)
    X_test_df = pd.DataFrame(X_test_sel, columns=feature_names)
    X_sample = X_test_df.sample(200, random_state=SEED).reset_index(drop=True)
    background = X_test_df.sample(100, random_state=SEED).reset_index(drop=True)

    explainer = shap.KernelExplainer(model.predict_proba, background)
    shap_values = np.array(explainer.shap_values(X_sample))
    if shap_values.ndim == 3: shap_values = shap_values[:, :, 1]

    plt.figure(figsize=(10, 6), dpi=300)
    shap.summary_plot(shap_values, X_sample, max_display=15, show=False)
    plt.title("SHAP Feature Importance", fontsize=14)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, 'Figure_SHAP_Summary.png'))
    plt.close()

    # ------------------------------------------------------------
    # PHASE 5: SAVING ARTIFACTS
    # ------------------------------------------------------------
    print("\n[5/5] Saving Final Pipeline for Deployment...")
    joblib.dump({
        "model": model,
        "scaler": scaler,
        "selector": selector,
        "features": feature_names.tolist()
    }, os.path.join(OUT_DIR, 'Q1_BetaLactam_Pipeline.pkl'))

    print("="*50)
    print(f"✅ SUCCESS! All publication files saved to: {OUT_DIR}")
    print("="*50)

if __name__ == "__main__":
    main()
```

```
Overwriting q1_betalactam_pipeline.py
```

```
!python q1_betalactam_pipeline.py
```

```
==================================================
🚀 INITIALIZING Q1 CHEMINFORMATICS PIPELINE 🚀
==================================================

[1/5] Loading and Sanitizing Data...
      -> Dropped 0 invalid SMILES.
      -> Stripped salts from 1004 molecules.
      -> Clean Dataset Size: 16750 molecules.

[2/5] Generating Features (Mordred & Morgan)...
100% 16750/16750 [1:39:44<00:00,  2.80it/s]
      -> Saving raw feature backup just in case...
      -> Cleaning Variance and Correlation...

[3/5] Training Final XGBoost Model...
Traceback (most recent call last):
  File "/content/q1_betalactam_pipeline.py", line 180, in <module>
    main()
  File "/content/q1_betalactam_pipeline.py", line 111, in main
    model.fit(X_train_sel, y_train_sub)
          ^^^^^^^^^^^^
NameError: name 'X_train_sel' is not defined
```

```
%%writefile q1_betalactam_recovery.py
"""
Beta-Lactam Pipeline - FAST RECOVERY SCRIPT
Skips the 1.5-hour feature generation by loading the backup.
"""

import os
import pandas as pd
import numpy as np
import random
import joblib
import warnings
import matplotlib.pyplot as plt
import shap

from rdkit import Chem
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.feature_selection import VarianceThreshold, SelectFromModel
from sklearn.metrics import roc_auc_score, matthews_corrcoef
from sklearn.linear_model import LinearRegression
from xgboost import XGBClassifier

# --- CONFIGURATION ---
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
warnings.filterwarnings("ignore")

OUT_DIR = '/content/drive/MyDrive/BetaLactam_Publication/'
DATA_PATH = '/content/drive/MyDrive/betalactam_expanded_500plus.csv'
BACKUP_PATH = os.path.join(OUT_DIR, 'RAW_features_backup.csv')

def main():
    print("="*50)
    print("🚀 INITIATING PIPELINE RECOVERY (SKIPPING CALCULATIONS) 🚀")
    print("="*50)

    # 1. Rebuild the Target Variable (y) instantly
    print("\n[1/4] Rebuilding target variable...")
    df_raw = pd.read_csv(DATA_PATH)
    valid_classes = []
    for index, row in df_raw.iterrows():
        mol = Chem.MolFromSmiles(str(row['smiles']))
        if mol is not None:
            valid_classes.append(row['Hydrolysis_Class'])
    y = pd.Series(valid_classes)

    # 2. Load the 1.5-hour features instantly!
    print("\n[2/4] Loading pre-calculated features from backup...")
    X = pd.read_csv(BACKUP_PATH)
    X.columns = X.columns.astype(str)

    print("      -> Cleaning Variance and Correlation...")
    vt = VarianceThreshold(0.001)
    X_vt = pd.DataFrame(vt.fit_transform(X), columns=X.columns[vt.get_support()])

    corr_matrix = X_vt.corr().abs()
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    to_drop = [col for col in upper.columns if any(upper[col] > 0.9)]
    X_clean = X_vt.drop(columns=to_drop)

    # --- THE MISSING SPLIT AND SCALE STEP ---
    print("      -> Splitting and Scaling Data...")
    X_train, X_ext, y_train, y_ext = train_test_split(X_clean, y, test_size=0.2, stratify=y, random_state=99)
    X_train_sub, X_test, y_train_sub, y_test = train_test_split(X_train, y_train, test_size=0.2, stratify=y_train, random_s

    scaler = StandardScaler()
    X_train_sub_scaled = scaler.fit_transform(X_train_sub)
    X_test_scaled = scaler.transform(X_test)
    X_ext_scaled = scaler.transform(X_ext)

    selector = SelectFromModel(XGBClassifier(n_estimators=200, random_state=SEED))
    selector.fit(X_train_sub_scaled, y_train_sub)

    X_train_sel = selector.transform(X_train_sub_scaled)
    X_test_sel = selector.transform(X_test_scaled)
    X_ext_sel = selector.transform(X_ext_scaled)

    # 3. Model Training
    print("\n[3/4] Training Final XGBoost Model...")
    best_params = {'subsample': 0.8, 'n_estimators': 500, 'max_depth': 8, 'learning_rate': 0.05, 'colsample_bytree': 1.0}
    model = XGBClassifier(**best_params, eval_metric='logloss', random_state=SEED)
    model.fit(X_train_sel, y_train_sub)

    y_pred = model.predict(X_test_sel)
    y_prob = model.predict_proba(X_test_sel)[:, 1]

    print(f"      -> Test ROC-AUC: {roc_auc_score(y_test, y_prob):.4f}")
    print(f"      -> External Set ROC-AUC: {roc_auc_score(y_ext, model.predict_proba(X_ext_sel)[:,1]):.4f}")
    print(f"      -> MCC: {matthews_corrcoef(y_test, y_pred):.4f}")

    # 4. Publication Figures
    print("\n[4/4] Generating Publication-Ready Figures...")

    # Williams Plot
    X_mat = np.array(X_train_sel)
    H = X_mat @ np.linalg.pinv(X_mat.T @ X_mat) @ X_mat.T
    leverage = np.diag(H)
    threshold = 3 * (X_train_sel.shape[1] / X_train_sel.shape[0])
```

```
    lr = LinearRegression()
    lr.fit(X_train_sel, y_train_sub)
    std_res = (y_train_sub - lr.predict(X_train_sel)) / np.std(y_train_sub - lr.predict(X_train_sel))

    plt.figure(figsize=(8, 6), dpi=300)
    plt.scatter(leverage, std_res, alpha=0.6, edgecolors='w', s=60)
    plt.axhline(3, color='r', linestyle='--')
    plt.axhline(-3, color='r', linestyle='--')
    plt.axvline(threshold, color='r', linestyle='--')
    plt.xlabel("Leverage (h)", fontsize=12)
    plt.ylabel("Standardized Residuals", fontsize=12)
    plt.title("Williams Plot: Applicability Domain", fontsize=14)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, 'Figure_Williams_Plot.png'))
    plt.close()

    # SHAP Plot
    feature_names = X_clean.columns[selector.get_support()].astype(str)
    X_test_df = pd.DataFrame(X_test_sel, columns=feature_names)
    X_sample = X_test_df.sample(200, random_state=SEED).reset_index(drop=True)
    background = X_test_df.sample(100, random_state=SEED).reset_index(drop=True)

    explainer = shap.KernelExplainer(model.predict_proba, background)
    shap_values = np.array(explainer.shap_values(X_sample))
    if shap_values.ndim == 3: shap_values = shap_values[:, :, 1]

    plt.figure(figsize=(10, 6), dpi=300)
    shap.summary_plot(shap_values, X_sample, max_display=15, show=False)
    plt.title("SHAP Feature Importance", fontsize=14)
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, 'Figure_SHAP_Summary.png'))
    plt.close()

    print("="*50)
    print(f"✅ SUCCESS! All publication files saved to: {OUT_DIR}")
    print("="*50)

if __name__ == "__main__":
    main()
```

```
Writing q1_betalactam_recovery.py
```

```
!python q1_betalactam_recovery.py
```

```
==================================================
🚀 INITIATING PIPELINE RECOVERY (SKIPPING CALCULATIONS) 🚀
==================================================

[1/4] Rebuilding target variable...

[2/4] Loading pre-calculated features from backup...
      -> Cleaning Variance and Correlation...
      -> Splitting and Scaling Data...

[3/4] Training Final XGBoost Model...
      -> Test ROC-AUC: 0.9151
      -> External Set ROC-AUC: 0.9191
      -> MCC: 0.6851

[4/4] Generating Publication-Ready Figures...
100% 200/200 [22:23<00:00,  6.72s/it]
==================================================
✅ SUCCESS! All publication files saved to: /content/drive/MyDrive/BetaLactam_Publication/
==================================================
```

```
# CELL: DECODING SHAP BITS TO CHEMICAL FRAGMENTS (FINAL FIX)
import pandas as pd
from rdkit import Chem
from rdkit.Chem import Draw
from rdkit.Chem import AllChem
from IPython.display import display
import warnings

# Hide RDKit deprecation warnings to keep the output clean
warnings.filterwarnings("ignore")

print("🔍 Decoding Morgan Fingerprint Bits 578 and 579...")

target_bits = [578, 579]
raw_df = pd.read_csv('/content/drive/MyDrive/betalactam_expanded_500plus.csv')
valid_mols = [Chem.MolFromSmiles(str(s)) for s in raw_df['smiles'].head(200) if Chem.MolFromSmiles(str(s)) is not None]

bit_images = []
```

```
bit_images = []
bit_labels = []

for bit in target_bits:
    found = False
    for mol in valid_mols:
        bit_info = {}
        # Mathematically calculate the fingerprint and extract the bit dictionary
        fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius=2, nBits=1024, bitInfo=bit_info)

        # If the target bit is present in this molecule
        if bit in bit_info:
            print(f"✅ Found Bit {bit} in molecule: {Chem.MolToSmiles(mol)}")
            img = Draw.DrawMorganBit(mol, bit, bit_info)
            bit_images.append(img)
            bit_labels.append(f"Bit_{bit}")
            found = True
            break

    if not found:
         print(f"⚠️ Could not find Bit {bit} in the first 200 molecules. Try scanning more.")

# Display and save each image individually
if bit_images:
    print("\nGenerating and saving structural fragment images...")
    for i, img in enumerate(bit_images):
        print(f"\n--- {bit_labels[i].replace('_', ' ')} ---")
        display(img)
        save_path = f'/content/drive/MyDrive/BetaLactam_Publication/Figure_{bit_labels[i]}.png'
        img.save(save_path)
        print(f"Saved to: {save_path}")
```